# Virtual Time Capsule

## Complete Project Documentation

A full-stack web application for creating, encrypting, and scheduling time-locked messages with geo-locking, collaborative capsules, real-time notifications, and shareable secret links.

**Developer:** Kairuppala Hemachandra
**GitHub:** github.com/KAIRUPPALA-HEMACHANDRA/virtual-time-capsule
**Live App:** https://virtual-time-capsule-ten.vercel.app
**API Docs:** https://virtual-time-capsule-iif2.onrender.com/api/docs

| Node.js | React | PostgreSQL | Prisma | Socket.io | JWT |
|---|---|---|---|---|---|

Version 1.0 | September 2026

# Table of Contents

# 1. Project Overview

## 1.1 What is Virtual Time Capsule?

Virtual Time Capsule is a full-stack web application that allows users to create digital time capsules — messages, images, voice recordings, and videos that are sealed and locked until a specific future date. When the scheduled time arrives, the capsule automatically unlocks and notifies the creator and recipients via email and real-time browser notifications.

The application transcends basic CRUD operations by incorporating advanced engineering concepts: end-to-end encryption where the server never sees plaintext (AES-256-GCM with PBKDF2 key derivation), geo-locked capsules that use the Haversine formula to verify physical location before unlocking, capsule chains that enable digital treasure hunts with sequential prerequisite checking, NLP-powered sentiment analysis that visualizes emotional patterns over time, collaborative group capsules where multiple users contribute sealed messages revealed simultaneously, real-time WebSocket notifications via Socket.io, shareable secret links for recipients without accounts, anonymous messaging mode for confessions and surprises, self-destructing messages that disappear after a single read, emoji reactions with creator notifications, a digital legacy mode that delivers messages upon user inactivity, and proof-of-creation certificates using SHA-256 cryptographic hashing.

The entire project was built using free and open-source tools, deployed on free-tier hosting services (Vercel, Render, Supabase), and costs zero money to run.

## 1.2 Problem Statement & Motivation

People want to preserve memories, send future messages to loved ones, create surprise reveals, and express feelings they cannot say directly. Existing solutions fall into two categories: basic reminder apps that lack emotional depth and security, or expensive enterprise solutions with unnecessary complexity. There was no platform that combined the emotional value of time capsules with production-grade engineering — scheduled job queues, end-to-end encryption, geo-spatial computation, real-time notifications, and collaborative features — all in a single, free, deployable application.

This project was motivated by several specific use cases that drove feature development:

**Secret confessions:** Anonymous capsules with hidden sender identity, shareable via WhatsApp links, with emoji reactions so the sender knows how the recipient felt.

**Birthday surprises:** Time-locked messages that automatically reveal at midnight on someone's birthday, with collaborative contributions from multiple friends.

**Treasure hunts:** Geo-locked capsule chains where each clue leads to a physical location, creating real-world scavenger hunts for dates, birthdays, or proposals.

**Digital legacy:** Messages to loved ones that are delivered if the creator becomes inactive, functioning as a digital will for important communications.

**Self-destructing secrets:** Messages that disappear after being read once, like a physical letter you burn after reading.

## 1.3 Key Metrics

| Metric | Value | Details |
|---|---|---|
| Total Features | 28 | 21 planned features + 7 bonus features |
| Lines of Code | ~8,000+ | Backend (~4,000) + Frontend (~4,000) |

| | | |
|---|---|---|
| Database Models | 7 | User, Capsule, Attachment, RefreshToken, Recipient, Contributor, Notification |
| Database Fields | 50+ | Capsule model alone has 22 fields |
| API Endpoints | 20+ | All documented with Swagger/OpenAPI |
| Automated Tests | 41 | Authentication (11), Capsule CRUD (16), Utilities (14) |
| Database Migrations | 12 | Incremental schema evolution |
| Security Layers | 6 | bcrypt, JWT rotation, AES-256-GCM, PBKDF2, Zod validation, rate limiting |
| Real-Time Events | 4 | Capsule unlock, collaboration invite, reaction, contributor joined |
| Development Time | ~10 days | Built incrementally across personal and office laptops |
| Total Cost | Zero | Entirely free and open-source tools and hosting |

# 2. System Architecture

## 2.1 High-Level Architecture

The application follows a three-tier architecture with a clear separation between the presentation layer (React frontend), business logic layer (Express.js API), and data layer (PostgreSQL database). Two additional communication layers operate alongside the main request-response cycle:

**Socket.io WebSockets:** A persistent bidirectional connection between each logged-in user's browser and the server. When events occur (capsule unlocks, reactions received, collaboration invites), the server pushes notifications instantly without the client polling. The connection is authenticated using the JWT access token in the handshake.

**pg-boss Job Queue:** A PostgreSQL-backed background job processor. When a capsule is created with a future unlock date, a job is scheduled in pg-boss. The job is stored as a row in PostgreSQL's own tables, meaning it survives server restarts. When the scheduled time arrives, pg-boss triggers a worker function that unlocks the capsule, creates notifications, and sends emails.

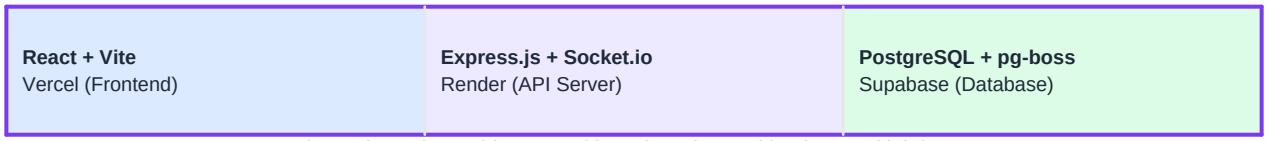


*Fig 1: Three-tier architecture with WebSocket and background job layers*

### Data Flow for Creating a Capsule:

1. User fills the create form in React (title, message, files, options)
2. React encrypts content client-side if encryption is enabled (Web Crypto API)
3. Axios sends a multipart POST request to /api/capsules with text fields + files
4. Express receives the request through Multer middleware (file parsing) and auth middleware (JWT verification)
5. Zod validates all input fields against the capsule schema
6. The capsule service runs sentiment analysis on the message text
7. A SHA-256 hash is generated for proof-of-creation
8. A unique share token is generated (crypto.randomBytes)
9. Prisma creates the capsule, attachments, and recipients in a single database transaction
10. pg-boss schedules an unlock job for the future date
11. The API returns the created capsule to the frontend
12. React navigates to the dashboard showing the new capsule with a countdown timer

## 2.2 Tech Stack — Backend

Each technology was chosen deliberately based on project requirements, Windows compatibility, and cost (free):

### Node.js + Express.js

Runtime and web framework. Express provides a minimal, flexible routing system with middleware support for authentication, validation, file uploads, rate limiting, and error handling. Chosen for its massive npm ecosystem and JavaScript full-stack consistency.

### PostgreSQL

Relational database chosen over MongoDB because time capsules have structured relationships — users create capsules, capsules have recipients, contributors, and attachments. Foreign keys enforce data integrity. JSONB columns provide flexibility for storing emoji reactions. PostgreSQL also powers the pg-boss job queue, eliminating the need for a separate Redis server.

### Prisma v6

Modern ORM that provides a declarative schema language (schema.prisma), auto-generated TypeScript-safe query API, and version-controlled migrations. Pinned to v6 because v7/v8 introduced breaking changes. The schema.prisma file serves as the single source of truth for the database structure, and Prisma's migration system tracks every schema change as a SQL file in version control.

### pg-boss

PostgreSQL-backed job queue. The original plan called for BullMQ + Redis, but Redis is difficult to install on Windows and adds infrastructure complexity. pg-boss stores scheduled jobs as rows in PostgreSQL's own tables, using the same database connection. This was a critical architectural decision that simplified deployment while maintaining reliable scheduled delivery with retry logic (3 retries, 60-second delay).

### Socket.io

WebSocket library for real-time bidirectional communication. When a capsule unlocks, a reaction is received, or a collaboration invite arrives, the server pushes a notification to the specific user's browser instantly. Each connected user joins a "room" named after their userId, enabling targeted message delivery. Authentication is verified in the connection handshake using the JWT access token.

### JWT + bcrypt

Authentication uses short-lived access tokens (15 minutes) and long-lived refresh tokens (7 days) with one-time-use rotation. Each refresh token includes a unique JTI (JWT ID) — a random string that ensures no two tokens are identical even when generated in the same millisecond. Passwords are hashed with bcrypt using 12 salt rounds, making brute-force attacks computationally impractical.

### Zod

Runtime schema validation library. Every API endpoint validates its inputs against a Zod schema before processing. This catches malformed data, SQL injection attempts, and type mismatches at the API boundary. Error messages are descriptive and user-friendly.

### Nodemailer

Email sending library using Gmail SMTP. Sends HTML-formatted notification emails to capsule creators and recipients when capsules unlock. Configured with 30-second connection timeout and 3-retry mechanism. Note: SMTP is blocked on Render's free tier; share links serve as the primary notification mechanism in production.

### Multer

Multipart form-data middleware for file uploads. Configured with 10MB file size limit, 5 files per capsule maximum, and file type validation (images, audio, video, PDF). Files are stored in a local /uploads directory in development and served as static files by Express.

### Swagger

Auto-generated interactive API documentation using swagger-jsdoc (extracts JSDoc comments from route files) and swagger-ui-express (renders the documentation UI). Every endpoint is documented with request parameters, response schemas, and authentication requirements. Accessible at /api/docs.

### Jest + Supertest

Testing framework with 41 automated tests. Jest runs the test suite with a custom configuration that mocks pg-boss (to avoid ESM import issues) and runs tests sequentially (to avoid database conflicts). Supertest makes HTTP requests to the Express app without starting the actual server.

## 2.3 Tech Stack — Frontend

### React 18 + Vite

Component-based UI library with Vite as the build tool. Vite provides instant hot module replacement during development and optimized production builds. React hooks (useState, useEffect, useContext, useRef) manage component state, side effects, and global context.

### React Router v6

Client-side routing with nested layouts, protected routes (redirect to login if not authenticated), and public routes (shared capsule view, landing page). A vercel.json rewrite rule ensures all routes are handled by React Router in production.

### Axios

HTTP client with request and response interceptors. The request interceptor attaches the JWT access token to every API call. The response interceptor catches 401 errors, automatically refreshes the access token using the refresh token cookie, and retries the original request — all transparently to the user.

### Context API (AuthContext + SocketContext)

Two React Context providers manage global state. AuthContext handles user authentication state, login/logout functions, and the loading state during initial auth checks. SocketContext manages the Socket.io connection lifecycle, notification state, and unread count.

### Recharts

Charting library for the emotion timeline on the profile page. Renders an interactive line chart showing sentiment scores over time, with custom tooltips displaying capsule titles, mood emojis, and dates.

### Web Crypto API

Browser-native cryptography API used for client-side AES-256-GCM encryption. The entire encryption/decryption process happens in the browser — the server never receives plaintext content or encryption keys. This is true zero-knowledge encryption.

### MediaRecorder API

Browser API for recording voice memos and video messages directly in the app. The VoiceRecorder component captures audio from the microphone, and the VideoRecorder component captures video from the camera with a live preview. Recordings are converted to File objects and uploaded alongside other attachments.

## 2.4 Design Decisions & Trade-offs

| Decision | Rationale & Trade-offs |
| --- | --- |
| pg-boss over BullMQ+Redis | BullMQ requires Redis, which is difficult to install on Windows. pg-boss uses PostgreSQL as its job store, eliminating an entire infrastructure dependency. Trade-off: slightly less performant for high-throughput scenarios, but perfectly adequate for a time capsule app where jobs are measured in days/months, not milliseconds. |
| PostgreSQL over MongoDB | Time capsules are inherently relational — users create capsules, capsules have recipients, contributors, and attachments. PostgreSQL's foreign keys, constraints, and ACID transactions ensure data integrity. MongoDB would require manual consistency management and denormalization. |

| | |
|---|---|
| Client-side encryption over server-side | Server-side encryption would be simpler to implement, but the server would have access to plaintext content. Client-side encryption using Web Crypto API ensures true zero-knowledge — even if the database is compromised, encrypted capsules are unreadable without the passphrase. Trade-off: the server cannot search or analyze encrypted content. |
| Multer local storage over cloud storage | Cloud storage (S3, Cloudinary) requires paid accounts or API keys. Local file storage with Multer is free and works immediately. Trade-off: files are stored on the server's filesystem and don't persist across Render deployments. For production, cloud storage would be recommended. |
| Share links over SMS/push notifications | SMS requires paid services (Twilio). Push notifications require service workers and HTTPS. Share links work everywhere — WhatsApp, Instagram, email, any messenger — for free. The recipient taps the link and sees a beautiful capsule page without needing an account. |

## 2.5 Project Structure

The project follows a monorepo structure with separate server/ and client/ directories. Each layer has a clear separation of concerns:

```
virtual-time-capsule/
  server/
    prisma/schema.prisma     # Single source of truth for DB schema
    src/
      config/                # db.js, env.js, queue.js, socket.js, swagger.js
      controllers/           # HTTP request handlers (thin layer)
      middleware/             # auth, validation, rateLimiter, upload, activityTracker
      services/              # Business logic (capsuleService, notificationService, contributorService)
      jobs/                  # Background workers (capsuleJobs, legacyJobs)
      utils/                 # Pure functions (email, geo, hash, sentiment, tokens, validators)
      routes/                # Express route definitions with Swagger JSDoc
      app.js                 # Express app assembly (middleware + routes)
      server.js              # HTTP server + Socket.io + pg-boss startup
    tests/                   # Jest test files + mocks
  client/
    src/
      components/            # 16 reusable components
      context/               # AuthContext, SocketContext
      pages/                 # 13 page components
      services/              # API client (api.js, authService.js, capsuleService.js)
      utils/                 # Client-side encryption
    vercel.json              # SPA routing for production
```

*The backend follows a Controller → Service → Prisma pattern. Controllers handle HTTP concerns (parsing requests, sending responses). Services contain business logic (validation rules, complex queries, notification creation). Prisma handles database access. This separation makes the code testable and maintainable.*

# 3. Database Design

## 3.1 Entity Relationship Overview

The database consists of 7 interconnected models managed through Prisma ORM. The central entity is the Capsule model, which has relationships to:

**User (creator):** One-to-many — each user can create multiple capsules

**Attachments:** One-to-many — each capsule can have up to 5 file attachments

**Recipients:** One-to-many — each capsule can have up to 10 delivery recipients

**Contributors:** One-to-many — each capsule can have multiple collaborators

**Capsule (self-referential):** One-to-one — each capsule can have a prerequisite capsule for chaining

**Notifications:** One-to-many via User — each user has their own notification stream

## 3.2 User Model

Stores registered user accounts with authentication credentials and activity tracking for legacy mode.

| Field | Type | Description |
|---|---|---|
| id | UUID (PK) | Auto-generated unique identifier |
| email | String (unique) | Login credential and notification target |
| password | String | bcrypt-hashed (12 salt rounds) — never stored in plaintext |
| name | String | Display name shown in capsules and notifications |
| lastActiveAt | DateTime (nullable) | Updated every 5 minutes by activity tracker middleware — used for legacy mode inactivity detection |
| createdAt | DateTime | Auto-set on registration |
| updatedAt | DateTime | Auto-updated on any change |

## 3.3 Capsule Model (22 fields)

The core entity with 22 fields covering all capsule features. This model demonstrates complex domain modeling — each field group enables a different feature:

### Core Fields:

| Field | Type | Description |
|---|---|---|
| id | UUID (PK) | Primary key |
| title | String (required) | Capsule title visible in listings |
| content | String (nullable) | Message text, or encrypted JSON blob for E2E encrypted capsules |
| status | Enum | LOCKED (sealed), UNLOCKED (time passed, ready to read), OPENED (viewed) |
| unlockAt | DateTime | When the capsule should become available |
| openedAt | DateTime? | When the content was first viewed |
| creatorId | UUID (FK) | Foreign key to User who created the capsule |

| | | |
|---|---|---|
| createdAt | DateTime | Creation timestamp (auto-set) |

### Feature Fields:

| Field | Type | Feature |
|---|---|---|
| isEncrypted | Boolean | End-to-end encryption (AES-256-GCM) |
| isPublic | Boolean | Appears on the public capsule wall when opened |
| isGeoLocked | Boolean | Requires GPS verification to unlock |
| latitude / longitude | Float? | GPS coordinates for geo-locked capsules |
| geoRadius | Int (default 100) | Unlock radius in meters (50-1000) |
| isLegacy | Boolean | Uses inactivity-triggered delivery instead of date |
| legacyDays | Int? | Days of inactivity before legacy delivery (30-365) |
| prerequisiteId | UUID? (self-FK) | Links to parent capsule for chains |
| contentHash | String? | SHA-256 hash for proof-of-creation certificate |
| shareToken | String? (unique) | Cryptographic token for shareable links |
| isAnonymous | Boolean | Hides creator identity on shared page |
| selfDestructAfterRead | Boolean | Content accessible only once via shared link |
| reactions | String? | JSON string storing emoji reactions from viewers |
| sentimentScore | Float? | NLP sentiment score (-1 to +1) |
| sentimentLabel | String? | Human-readable mood: happy, sad, hopeful, melancholic, neutral |

## 3.4 Supporting Models

| Model | Fields | Purpose |
|---|---|---|
| Attachment | id, filename, originalName, mimetype, size, path, capsuleId | File uploads (images, audio, video, PDF) linked to capsules via foreign key with cascade delete |
| RefreshToken | id, token (unique), userId, expiresAt | JWT refresh tokens stored in the database for one-time-use rotation. Old tokens are deleted on refresh. |
| Recipient | id, email, capsuleId, notified, viewedAt | Delivery targets for capsule notifications. The notified flag prevents duplicate emails. |
| Contributor | id, capsuleId, userId, email, status, content, invitedAt, joinedAt | Collaborative capsule participants. Status transitions: pending > accepted > contributed. Content is hidden until capsule unlocks. Unique constraint on (capsuleId, email) prevents duplicate invitations. |
| Notification | id, type, title, message, read, capsuleId, userId | Persistent notification records. Types: capsule_unlocked, capsule_invite, contributor_joined, capsule_reaction. Pushed via Socket.io in real-time and persisted for history. |

## 3.5 Migration History

The database schema evolved through 12 incremental Prisma migrations. Each migration is a SQL file stored in version control, providing a complete audit trail of how the schema grew as features were added:

| # | Migration Name | What It Added | Phase |
|---|---|---|---|
| 1 | init | User and Capsule base tables, RefreshToken | Foundation |
| 2 | add_attachments | Attachment model with cascade delete | Media |
| 3 | add_content_hash | SHA-256 hash field on Capsule | Security |
| 4 | add_sentiment | sentimentScore and sentimentLabel | Analytics |
| 5 | add_geo_fields | isGeoLocked, latitude, longitude, geoRadius | Geo |
| 6 | add_capsule_chains | prerequisiteId self-referential FK | Chains |
| 7 | add_recipients | Recipient model | Social |
| 8 | add_legacy_mode | isLegacy, legacyDays, User.lastActiveAt | Legacy |
| 9 | add_notifications | Notification model | Real-time |
| 10 | add_contributors | Contributor model with unique constraint | Collaboration |
| 11 | add_share_token | shareToken unique field | Sharing |
| 12 | add_missing_fields | isAnonymous, selfDestructAfterRead, reactions | Bonus |

# 4. Authentication System

The authentication system implements the OAuth 2.0 refresh token rotation pattern — the industry standard for secure web application authentication. This is significantly more secure than simple JWT-based auth.

## 4.1 Registration Flow

When a user registers, the following steps occur:

1. The frontend sends a POST request with name, email, and password
2. Zod validates the input: name (min 2 chars), email (valid format), password (min 8 chars, must contain uppercase, lowercase, and number)
3. The service checks if the email already exists (returns 409 Conflict if duplicate)
4. The password is hashed with bcrypt using 12 salt rounds
5. A new User record is created in the database
6. An access token (15min) and refresh token (7 days) are generated
7. The refresh token is stored in the database and sent as an HTTP-only cookie
8. The access token is returned in the response body
9. The frontend stores the access token in localStorage and redirects to the dashboard

### API Example:

```
POST /api/auth/register

Request Body:
{
  "name": "Hemachandra",
  "email": "hema@test.com",
  "password": "Test1234"
}

Response (201):
{
  "status": "success",
  "accessToken": "eyJhbGciOiJIUzI1NiIs...",
  "data": {
    "user": {
      "id": "ab471ba4-b3b6-4959-91af-f2de185354c0",
      "name": "Hemachandra",
      "email": "hema@test.com"
    }
  }
}

Set-Cookie: refreshToken=eyJhbG...; HttpOnly; Path=/; Max-Age=604800
```

## 4.2 Login & Token Rotation

Login verifies credentials and implements token rotation — each refresh token can only be used once. When refreshed, the old token is deleted and a new pair is issued. This prevents token replay attacks.

### Token Rotation Code:

```
async function refreshAccessToken(token) {
  // Find the token in the database
  const storedToken = await prisma.refreshToken.findUnique({ where: { token } });
  if (!storedToken) throw new AppError("Refresh token not found", 401);
  if (storedToken.expiresAt < new Date()) throw new AppError("Token expired", 401);
```

```
  // Delete the old token (one-time use)
  await prisma.refreshToken.delete({ where: { id: storedToken.id } });

  // Generate new pair
  const accessToken = generateAccessToken(storedToken.userId);
  const refreshToken = generateRefreshToken(storedToken.userId);

  // Store new refresh token
  await prisma.refreshToken.create({ data: { token: refreshToken, userId: storedToken.userId, expiresAt: .
.. } });

  return { accessToken, refreshToken };
}
```

## 4.3 Protected Routes

The auth middleware intercepts every request to protected endpoints:

```
function protect(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader?.startsWith("Bearer ")) throw new AppError("Not authenticated", 401);

  const token = authHeader.split(" ")[1];
  const decoded = verifyAccessToken(token); // jwt.verify()
  req.user = { userId: decoded.userId };
  next();
}
```

## 4.4 Password Change

When a user changes their password, ALL existing refresh tokens for that user are deleted. This immediately invalidates all active sessions across all devices — the user must log in again everywhere with the new password. This is a security best practice that prevents old sessions from persisting after a password change.

# 5. Capsule Management

## 5.1 Create Capsule (Full Flow)

Capsule creation is the most complex operation in the application. A single API call handles text content, file uploads, encryption, recipient management, sentiment analysis, hash generation, token creation, and job scheduling. The endpoint accepts multipart/form-data (via Multer) to handle simultaneous text fields and file uploads.

### What happens inside capsuleService.createCapsule():

```
async function createCapsule(userId, data, files) {
  // 1. Sentiment analysis
  const sentiment = analyzeSentiment(data.content || "");

  // 2. SHA-256 hash for proof-of-creation
  const contentHash = generateContentHash(data.title, data.content, new Date());

  // 3. Create capsule with all related data in one transaction
  const capsule = await prisma.capsule.create({
    data: {
      title: data.title,
      content: data.content,      // Already encrypted if E2E is enabled
      unlockAt: new Date(data.unlockAt),
      isPublic: data.isPublic,
      creatorId: userId,
      isEncrypted: data.isEncrypted,
      isAnonymous: data.isAnonymous,
      selfDestructAfterRead: data.selfDestructAfterRead,
      shareToken: generateShareToken(),  // crypto.randomBytes(12)
      isGeoLocked: data.isGeoLocked,
      latitude: parseFloat(data.latitude),
      longitude: parseFloat(data.longitude),
      geoRadius: parseInt(data.geoRadius),
      prerequisiteId: data.prerequisiteId,
      isLegacy: data.isLegacy,
      legacyDays: parseInt(data.legacyDays),
      contentHash,
      sentimentScore: sentiment.score,
      sentimentLabel: sentiment.label,
      recipients: { create: emails.map(e => ({ email: e })) },
      attachments: { create: files.map(f => ({ ... })) },
    },
  });

  // 4. Schedule unlock job (only for non-legacy capsules)
  if (!capsule.isLegacy) {
    const boss = getQueue();
    await scheduleCapsuleUnlock(boss, capsule.id, capsule.unlockAt);
  }

  return capsule;
}
```

## 5.2 Content Visibility Rules

| Status | Content Visible? | Attachments Visible? | Rule |
|---|---|---|---|
| LOCKED | No (returns null) | No | Time has not passed yet |

| | | | |
|---|---|---|---|
| LOCKED (time passed, geo-locked) | No | No | Must check location first |
| LOCKED (time passed, chained) | No | No | Prerequisite must be opened first |
| UNLOCKED | Yes | Yes | Time passed and all conditions met |
| OPENED | Yes | Yes | User has already viewed the content |
| OPENED (self-destruct, via share) | Error 410 | No | Content destroyed after first shared view |

## 5.3 Status Lifecycle

LOCKED → UNLOCKED: Triggered by pg-boss job, auto-unlock on dashboard load, or geo-check endpoint. Requires: time passed + not geo-locked (or geo-checked) + prerequisite met.

UNLOCKED → OPENED: Triggered when the owner views the capsule (except self-destruct capsules, which stay UNLOCKED until viewed via share link).

# 6. Scheduled Delivery System

## 6.1 Why pg-boss over BullMQ + Redis

The original architecture plan specified BullMQ with Redis for job scheduling. During development, this was changed to pg-boss for three reasons:

**Windows compatibility:** Redis does not have an official Windows installer. Options like WSL or Memurai add complexity and potential failure points. pg-boss uses the existing PostgreSQL connection — no additional software installation.

**Deployment simplicity:** One fewer service to configure, monitor, and pay for. Supabase provides PostgreSQL; no separate Redis hosting needed.

**Adequate performance:** Time capsule jobs are measured in days/months, not milliseconds. pg-boss polls every 30 seconds, which is more than sufficient. BullMQ's sub-second latency would be overkill.

## 6.2 Job Scheduling Flow

```
// When a capsule is created:
await boss.send("capsule-unlock", { capsuleId }, {
  id: capsuleId,        // Job ID must be UUID format
  startAfter: unlockAt, // Future date
  retryLimit: 3,        // Retry 3 times on failure
  retryDelay: 60,       // 60 seconds between retries
});

// Worker function (runs when time arrives):
async function handleCapsuleUnlock(job) {
  const capsule = await prisma.capsule.findUnique(...);

  // Check conditions:
  if (capsule.isGeoLocked) return;        // Skip geo-locked
  if (prerequisite not met) return;        // Skip chained

  // Unlock
  await prisma.capsule.update({ data: { status: "UNLOCKED" } });

  // Notify
  await createNotification({ ... });       // Socket.io push
  await sendCapsuleUnlockEmail(user, capsule);  // Email

  // Notify recipients
  for (const recipient of capsule.recipients) {
    await sendEmail({ to: recipient.email, ... });
  }
}
```

## 6.3 Auto-Unlock Mechanism

As a safety net, the auto-unlock function runs every time a user loads their dashboard. It finds all LOCKED capsules whose unlock time has passed (excluding geo-locked and chained capsules) and unlocks them immediately. This catches any capsules that the pg-boss job missed (e.g., if the server was down when the job was due).

## 6.4 Legacy Mode Daily Check

A separate pg-boss recurring job runs daily at midnight (cron: 0 0 * * *). It finds all legacy capsules, checks each creator's lastActiveAt against their configured legacyDays threshold, and unlocks capsules for inactive users. Recipients receive notification emails with share links.

# 7. End-to-End Encryption

This is the most technically sophisticated security feature in the application. It implements true zero-knowledge encryption — the server is a storage medium only. It never sees, processes, or has the ability to decrypt capsule content.

## 7.1 Why Client-Side Encryption

Server-side encryption (encrypting on the server before storing) would be simpler but fundamentally weaker. The server would hold both the encrypted data and the encryption key — meaning a database breach or a compromised server would expose all content. Client-side encryption ensures that even if an attacker gains full access to the server and database, they cannot read encrypted capsules without each capsule's unique passphrase.

## 7.2 Encryption Flow (AES-256-GCM)

AES-256-GCM (Advanced Encryption Standard, 256-bit key, Galois/Counter Mode) is a symmetric encryption algorithm used by governments and military organizations worldwide. GCM mode provides both confidentiality (nobody can read the data) and authenticity (nobody can tamper with the data without detection).

```
async function encryptContent(plaintext, passphrase) {
  // Generate random salt (16 bytes) and IV (12 bytes)
  const salt = crypto.getRandomValues(new Uint8Array(16));
  const iv = crypto.getRandomValues(new Uint8Array(12));

  // Derive 256-bit key from passphrase using PBKDF2
  const keyMaterial = await crypto.subtle.importKey("raw", encode(passphrase), "PBKDF2", false, ["deriveKe
y"]);
  const key = await crypto.subtle.deriveKey(
    { name: "PBKDF2", salt, iterations: 100000, hash: "SHA-256" },
    keyMaterial,
    { name: "AES-GCM", length: 256 },
    false,
    ["encrypt"]
  );

  // Encrypt
  const encrypted = await crypto.subtle.encrypt({ name: "AES-GCM", iv }, key, encode(plaintext));

  // Return Base64-encoded values for server storage
  return { encryptedData: toBase64(encrypted), salt: toBase64(salt), iv: toBase64(iv) };
}
```

## 7.3 Key Derivation (PBKDF2)

A passphrase like "our secret place" cannot be used directly as an encryption key — it lacks sufficient entropy (randomness). PBKDF2 (Password-Based Key Derivation Function 2) converts the passphrase into a cryptographically strong 256-bit key by:

- Mixing the passphrase with a random salt (prevents identical passphrases from producing identical keys)
- Running 100,000 iterations of SHA-256 hashing (makes brute-force attacks extremely slow — each guess takes ~100ms)
- Producing a fixed-length 256-bit output suitable for AES-256

## 7.4 Decryption Flow

Decryption reverses the process: the same passphrase + stored salt produces the same key through PBKDF2, then AES-GCM decrypts using the stored IV. If the passphrase is wrong, GCM's authentication tag verification fails and

an error is thrown — no partial or corrupted data is ever returned.

## 7.5 Security Guarantees

| Guarantee | How It's Achieved |
|---|---|
| Confidentiality | AES-256-GCM encryption — computationally infeasible to break |
| Integrity | GCM authentication tag — any tampering is detected |
| Zero Knowledge | Encryption/decryption happens in the browser — server never sees plaintext or keys |
| Unique Ciphertext | Random salt and IV ensure the same message + passphrase produces different output each time |
| Brute-Force Resistance | PBKDF2 with 100,000 iterations — each passphrase guess takes ~100ms, making dictionary attacks impractical |

# 8. Geo-Locked Capsules

A capsule can be pinned to a specific GPS location with a configurable radius (50-1000 meters). The capsule will not unlock — even after its scheduled time passes — unless the viewer is physically at the right location. This is implemented using the Haversine formula for geospatial distance calculation.

## 8.1 The Haversine Formula

The Haversine formula calculates the shortest distance between two points on a sphere (Earth) given their latitude and longitude coordinates. It accounts for Earth's curvature and is accurate for distances up to several hundred kilometers.

```
function haversineDistance(lat1, lon1, lat2, lon2) {
  const R = 6371000; // Earth's radius in meters
  const toRad = (deg) => (deg * Math.PI) / 180;
  const dLat = toRad(lat2 - lat1);
  const dLon = toRad(lon2 - lon1);
  const a = Math.sin(dLat/2) * Math.sin(dLat/2) +
    Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) *
    Math.sin(dLon/2) * Math.sin(dLon/2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
  return Math.round(R * c); // Distance in meters
}
```

*This is the same mathematical formula used by Google Maps, Uber, and every location-based service. The function returns the distance in meters, which is compared against the capsule's configured radius.*

## 8.2 Frontend Geolocation API

The browser's navigator.geolocation.getCurrentPosition() API requests the user's GPS coordinates (with their permission). High accuracy mode is enabled for better results. A 10-second timeout prevents the UI from hanging if GPS is unavailable.

## 8.3 Geo-Check Endpoint

POST /api/capsules/:id/geo-check accepts latitude and longitude, calculates the Haversine distance, and returns whether the user is within range. If within range AND the unlock time has passed, the capsule is unlocked in the same request.

---

# 9. Capsule Chains

Capsule chains enable sequential unlocking — each capsule in a chain only becomes available after the previous one has been opened. This is implemented through a self-referential foreign key (prerequisiteId) on the Capsule model.

Before unlocking a chained capsule, the service checks:

```
const prerequisiteMet = !capsule.prerequisiteId ||
  (capsule.prerequisite &&
   ["UNLOCKED", "OPENED"].includes(capsule.prerequisite.status));
```

Combined with geo-locking, capsule chains enable real-world treasure hunts. Example: A series of 3 capsules where each one is geo-locked to a different location and chained to the previous. The recipient must physically travel to location 1, open capsule 1 (which gives a clue), travel to location 2, open capsule 2, and so on until the

final reveal.

## 10. Sentiment Analysis & Emotion Timeline

When a capsule is created, the message text is analyzed using the sentiment npm package — a lexicon-based NLP library. It assigns a numerical score (-1 to +1, where -1 is very negative and +1 is very positive) and maps it to a human-readable label with an emoji:

| Score Range | Label | Emoji |
|---|---|---|
| > 0.5 | happy | smile emoji |
| 0.1 to 0.5 | hopeful | sun emoji |
| -0.1 to 0.1 | neutral | neutral emoji |
| -0.5 to -0.1 | melancholic | pensive emoji |
| < -0.5 | sad | cry emoji |

On the profile page, all capsule sentiments are plotted as an interactive Recharts line chart. Hovering over a data point shows the capsule title, mood emoji, sentiment score, and creation date. A reference line at y=0 separates positive from negative sentiments. Below the chart, a legend shows the count of each mood category.

# 11. Digital Legacy Mode

Digital Legacy Mode transforms a time capsule from a date-triggered delivery into an inactivity-triggered delivery — a "digital will." If the creator stops logging in for their configured period (30 days to 1 year), their legacy capsules are automatically unlocked and delivered to recipients.

The system has two components:

**Activity Tracker Middleware:** Updates User.lastActiveAt on every authenticated API request. To avoid excessive database writes, updates are throttled to once every 5 minutes using an in-memory cache (Map). Simply logging in resets the inactivity timer.

**Daily Check Job:** A pg-boss recurring job (cron: "0 0 * * *") runs at midnight every day. It queries all LOCKED legacy capsules, calculates days since each creator's last activity, and unlocks capsules where the inactivity exceeds the threshold. Recipients receive email notifications with share links.

---

# 12. Recipient System & Email Notifications

Users can add up to 10 recipient email addresses when creating a capsule. The RecipientInput component on the frontend provides an email tag interface — type an email, press Enter or click Add, and it appears as a removable tag. Input validation ensures proper email format and prevents duplicates.

When a capsule unlocks, the auto-unlock function iterates through all recipients with notified=false and sends an HTML email containing the capsule title and a clickable share link. After sending, the notified flag is set to true to prevent duplicate emails.

The email template uses a dark-themed HTML design consistent with the app's visual identity. The share link in the email points to the production Vercel URL, allowing recipients to view the capsule without creating an account.

# 13. Collaborative Group Capsules

Collaborative capsules allow multiple users to contribute sealed messages to a single capsule. All contributions are hidden from everyone — including the creator — until the capsule unlocks.

## 13.1 Invitation Flow

1. Creator views their capsule and enters a contributor's email in the invite form
2. POST /api/collaborate/invite creates a Contributor record with status "pending"
3. If the email belongs to a registered user, they receive a Socket.io notification ("You've been invited to contribute to...")
4. The contributor visits /invitations and sees the pending invitation
5. They click Accept, which changes status to "accepted" and notifies the creator

## 13.2 Contribution Flow

1. An accepted contributor views the capsule and sees an "Add My Contribution" button
2. They write their message and click "Seal My Contribution"
3. POST /api/collaborate/:id/contribute saves the content and changes status to "contributed"
4. The contribution content is stored but hidden (the API returns null for content when the capsule is LOCKED)

## 13.3 Reveal on Unlock

When the capsule unlocks, all contributions become visible. The ViewCapsule page and SharedCapsule page both display each contributor's name and message. This creates a powerful emotional moment — everyone sees what others secretly wrote, all at once.

---

# 14. Real-Time Notifications (Socket.io)

## 14.1 Connection & Authentication

When a user logs in, the SocketContext creates a Socket.io connection to the server. The JWT access token is sent in the handshake auth parameter. The server's authentication middleware verifies the token and assigns the socket to a room named after the user's ID:

```
// Server-side
io.use((socket, next) => {
  const token = socket.handshake.auth.token;
  const decoded = verifyAccessToken(token);
  socket.userId = decoded.userId;
  next();
});

io.on("connection", (socket) => {
  socket.join(socket.userId); // User-specific room
});

// Sending to a specific user:
function emitToUser(userId, event, data) {
  io.to(userId).emit(event, data);
}
```

## 14.2 Notification Types

| Type | Event | Trigger |
|---|---|---|
| capsule_unlocked | notification | Capsule auto-unlock or time-based unlock |
| capsule_invite | notification | Creator invites a contributor |
| contributor_joined | notification | Contributor accepts an invitation |
| capsule_reaction | notification | Someone reacts to a shared capsule with an emoji |

## 14.3 Frontend Bell Component

The NotificationBell component in the navbar shows a bell icon with a red badge displaying the unread count. Clicking it opens a dropdown with recent notifications, each showing the message text and relative time ("2m ago", "1h ago"). Clicking a notification navigates to the relevant capsule and marks it as read via PATCH /api/notifications/:id/read. A "Mark all read" button clears all unread badges.

# 15. Shareable Links & Anonymous Mode

Every capsule is assigned a unique share token at creation — a cryptographically random 16-character URL-safe string generated by crypto.randomBytes(12).toString("base64url"). The share URL format is:

```
https://virtual-time-capsule-ten.vercel.app/shared/AiTRKrs8rCt_dSvz
```

Anyone with this link can view the capsule content after it unlocks — no account required. This is the primary sharing mechanism, designed for WhatsApp, Instagram DMs, and other messengers. The shared page includes:

- A beautiful full-page reveal with "A Message From The Past" header
- Creator attribution (or "From someone special" with mask emoji if anonymous)
- The capsule content, attachments, and contributor messages
- A countdown timer if the capsule hasn't unlocked yet
- A row of 8 emoji reaction buttons
- Self-destruct warning banner if enabled

**Anonymous Mode:** When isAnonymous is true, the shared page displays "From someone special" with a mask emoji instead of the creator's name. The API response replaces creator.name with "Anonymous" on the shared endpoint. The mystery of not knowing who sent the message is what makes anonymous confessions compelling for young social media users.

---

# 16. Self-Destruct Capsules

When selfDestructAfterRead is true, the capsule content is viewable only once via the shared link. Implementation:

- **First shared view:** Content is displayed normally. The capsule status is changed to OPENED.
- **Second shared view:** The controller checks if status is OPENED AND selfDestructAfterRead is true. If so, it returns HTTP 410 Gone with "This capsule has self-destructed after being read."
- **Owner view:** The owner can always view the capsule without triggering self-destruct. This is enforced by the condition !capsule.selfDestructAfterRead in the status-change logic of getCapsuleById.

---

# 17. Emoji Reactions

The shared capsule page displays 8 emoji reaction buttons: heart, love-eyes, laugh, cry, pleading, mind-blown, fire, clap. When tapped, a POST to /api/shared/:token/react stores the reaction as a JSON array in the capsule's reactions field and creates a Socket.io notification for the creator. Each visitor can react only once (controlled by React state).

The creator receives a notification like: "heart emoji Reaction! Someone reacted heart emoji to your capsule 'Secret Message'". This closes the emotional loop — the sender knows how the recipient felt, even without knowing who they are.

# 18. Additional Features

## 18.1 Public Capsule Wall

A publicly accessible page (/wall) showing all capsules marked as public that have been opened. No authentication required. Each capsule card shows the title, message content, creator name, and creation date. This creates a community dimension where users can browse messages from the past.

## 18.2 Proof-of-Creation Certificates

Every capsule gets a SHA-256 hash of (title + content + timestamp) stored at creation. The /verify/:id page displays this hash as a "Proof-of-Creation Certificate" — cryptographic evidence that the content existed at a specific point in time. A separate /verify/:id/check endpoint accepts content and verifies it against the stored hash. This is useful for predictions, ideas, or creative work where proving the creation date matters.

## 18.3 Voice & Video Recording

The VoiceRecorder and VideoRecorder components use the browser's MediaRecorder API to record audio and video directly in the app. VoiceRecorder captures microphone input with a timer display and playback preview. VideoRecorder shows a live camera preview (mirrored for natural selfie view) with a red REC indicator during recording. Both components convert the recorded blob to a File object that's uploaded alongside other attachments using the existing Multer infrastructure.

## 18.4 Profile & Analytics Dashboard

The profile page shows user information (name, email, user ID), a password change form, and the emotion timeline chart. The chart visualizes sentiment analysis data across all capsules, with mood distribution counts at the bottom (e.g., "happy: 3, sad: 2, neutral: 5").

## 18.5 Search & Sort

The dashboard provides a search bar that filters capsules by title (case-insensitive), status filter tabs (All, Locked, Unlocked, Opened) with counts, and a sort dropdown (Newest first, Oldest first, Opening soon, A-Z, Z-A). All filtering and sorting happen client-side using useMemo for performance.

## 18.6 Responsive Design

The entire application is responsive. The navbar collapses into a hamburger menu on mobile screens. The capsule grid adjusts from 3 columns (desktop) to 2 (tablet) to 1 (mobile). The create form, view page, and shared capsule page all adapt to narrow viewports. Tested using Chrome DevTools device toolbar.

# 19. API Reference (Complete)

Full interactive documentation is available at /api/docs (Swagger UI). Below is the complete endpoint listing:

## Authentication

| Method | Endpoint | Auth | Description |
|---|---|---|---|
| POST | /api/auth/register | No | Create account (name, email, password) |
| POST | /api/auth/login | No | Login (returns access token + refresh cookie) |
| POST | /api/auth/refresh | Cookie | Refresh access token using HTTP-only cookie |
| POST | /api/auth/logout | Cookie | Invalidate refresh token |
| GET | /api/auth/me | Bearer | Get current user profile |
| PATCH | /api/auth/change-password | Bearer | Change password (invalidates all sessions) |

## Capsules

| Method | Endpoint | Description |
|---|---|---|
| POST | /api/capsules | Create capsule (multipart/form-data with files) |
| GET | /api/capsules | List user's capsules (locked content hidden) |
| GET | /api/capsules/:id | Get single capsule with auto-unlock logic |
| PATCH | /api/capsules/:id | Update a locked capsule (title, content, date) |
| DELETE | /api/capsules/:id | Delete capsule, attachments, and recipients |
| DELETE | /api/capsules/:id/attachments/:aid | Delete a single attachment |
| POST | /api/capsules/:id/geo-check | Submit GPS coordinates for geo-unlock |
| GET | /api/capsules/stats/sentiment | Get emotion timeline data |

## Collaboration

| Method | Endpoint | Description |
|---|---|---|
| POST | /api/collaborate/invite | Invite contributors by email |
| POST | /api/collaborate/:id/accept | Accept collaboration invitation |
| POST | /api/collaborate/:id/contribute | Add sealed contribution text |
| GET | /api/collaborate/:id/contributors | List contributors with status |
| GET | /api/collaborate/invitations | Get pending invitations for current user |

## Notifications

| Method | Endpoint | Description |
|---|---|---|
| GET | /api/notifications | Get all notifications with unread count |
| PATCH | /api/notifications/:id/read | Mark a single notification as read |
| PATCH | /api/notifications/read-all | Mark all notifications as read |

## Public (No Auth Required)

| Method | Endpoint | Description |
|---|---|---|
| GET | /api/public/capsules | Public capsule wall (opened public capsules) |
| GET | /api/verify/:id | Proof-of-creation certificate |
| POST | /api/verify/:id/check | Verify content against stored hash |
| GET | /api/shared/:token | View shared capsule (no login needed) |
| POST | /api/shared/:token/react | Add emoji reaction to shared capsule |
| GET | /api/health | Server health check with timestamp |
| GET | /api/docs | Interactive Swagger API documentation |

# 20. Security Implementation

| Layer | Measure | Details |
|---|---|---|
| Password | bcrypt (12 rounds) | Each round doubles computation time. 12 rounds = ~250ms per hash. Makes brute-force attacks impractical. |
| Authentication | JWT access + refresh tokens | Access: 15min expiry. Refresh: 7 days, one-time use with rotation. Each token has unique JTI. |
| Cookie Security | HTTP-only, Secure | Refresh tokens in HTTP-only cookies — inaccessible to JavaScript (prevents XSS token theft). |
| Encryption | AES-256-GCM + PBKDF2 | Client-side encryption with 100,000 PBKDF2 iterations. Server never sees plaintext. |
| Input Validation | Zod schemas | Every endpoint validates inputs. Rejects malformed data before business logic. |
| Rate Limiting | express-rate-limit | 100 req/15min general, 10 req/15min for auth routes. Prevents brute-force attacks. |
| HTTP Headers | Helmet.js | X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, and other security headers. |
| CORS | Origin whitelist | Only accepts requests from the specific frontend origin. Credentials: true for cookies. |
| Data Integrity | SHA-256 hashing | Content hash generated at creation. Proves content existed at a specific time without revealing it. |
| Trust Proxy | Express setting | app.set("trust proxy", 1) — correctly identifies client IPs behind Render's reverse proxy. |

# 21. Testing Suite

The application includes 41 automated tests across 3 test suites:

## Authentication Tests (11 tests)

| Test | What It Verifies |
|---|---|
| Register successfully | 201 status, user object returned, no password in response |
| Reject duplicate email | 409 status on existing email |
| Reject weak password | 400 status on "123" |
| Reject invalid email | 400 status on "not-an-email" |
| Reject missing name | 400 status when name is omitted |
| Login with correct credentials | 200 status, access token returned |
| Reject wrong password | 401 status |
| Reject non-existent email | 401 status |
| Return profile with valid token | 200 status, user data returned |
| Reject request without token | 401 status |
| Reject invalid token | 401 status on "Bearer invalidtoken123" |

## Capsule Tests (16 tests)

| Test | What It Verifies |
|---|---|
| Create capsule successfully | 201 status, LOCKED status, contentHash generated |
| Reject capsule without title | Non-201 status |
| Reject without auth | 401 status |
| Create geo-locked capsule | isGeoLocked=true, correct lat/lon stored |
| Create legacy capsule | isLegacy=true, legacyDays=180 |
| Return all user capsules | Array returned, length > 0 |
| Hide locked capsule content | Content null for locked capsules |
| Reject list without auth | 401 status |
| Return single capsule | Correct ID returned |
| Return 404 for non-existent | 404 status on invalid UUID |
| Update locked capsule | 200 status, title changed |
| Reject update from another user | 403 status |
| Delete own capsule | 200 status, "deleted" message |
| Reject delete without auth | 401 status |

| | |
|---|---|
| Return server health | 200 status, "running" message |
| Return 404 for unknown routes | 404 status on /api/nonexistent |

## Utility Tests (14 tests)

| Test | What It Verifies |
|---|---|
| Consistent hash for same input | Same title+content+date produces identical SHA-256 |
| Different hashes for different content | Changed content produces different hash |
| 64-character hex string | Hash matches /^[a-f0-9]{64}$/ pattern |
| Verify matching content | Returns true for correct content |
| Reject non-matching content | Returns false for tampered content |
| Calculate distance between points | Hyderabad-Mumbai: 600-650km |
| Return 0 for same location | Same coordinates: 0 meters |
| Approve unlock within radius | 0m distance, 100m radius: true |
| Reject unlock outside radius | ~1km distance, 100m radius: false |
| Detect happy sentiment | "so happy and excited" scores > 0.5 |
| Detect sad sentiment | "terrible and awful" scores < -0.5 |
| Detect neutral sentiment | "went to the store" scores near 0 |
| Handle empty text | Empty string: score 0, label neutral |
| Return emoji with result | Emoji property defined in result |

### Testing Infrastructure:

pg-boss uses ESM imports which Jest cannot parse. A custom mock at tests/mocks/queue.js replaces the pg-boss module during testing. Jest is configured with moduleNameMapper in jest.config.js to redirect queue imports to the mock. Tests run sequentially (maxWorkers: 1) to avoid database conflicts, and each suite cleans up test data in beforeAll/afterAll hooks.

# 22. Deployment Guide

## 22.1 Production Architecture

| Layer | Service | Plan | URL |
|---|---|---|---|
| Frontend | Vercel | Free | https://virtual-time-capsule-ten.vercel.app |
| Backend | Render | Free | https://virtual-time-capsule-iif2.onrender.com |
| Database | Supabase | Free | PostgreSQL, Singapore region |

## 22.2 Supabase Setup

1. Sign up at supabase.com with GitHub
2. Create new project with name, password, and Singapore region
3. Click Connect > ORM > Prisma to get DATABASE_URL and DIRECT_URL
4. DATABASE_URL uses port 6543 with ?pgbouncer=true (connection pooling)
5. DIRECT_URL uses port 5432 without pgbouncer (used for migrations only)

## 22.3 Render Deployment

1. Sign up at render.com with GitHub
2. Create new Web Service, connect the virtual-time-capsule repo
3. Set root directory to "server", runtime to Node
4. Build command: npm install && npm install @prisma/client@6 --save-exact && npx prisma@6 generate
5. Start command: npx prisma@6 migrate deploy && node src/server.js
6. Add 15 environment variables (see section 22.5)

## 22.4 Vercel Deployment

1. Sign up at vercel.com with GitHub
2. Import the virtual-time-capsule repo
3. Set root directory to "client"
4. Add environment variable: VITE_API_URL = [Render URL]/api
5. Deploy — Vercel auto-detects Vite and builds
6. Update CLIENT_URL in Render with the Vercel URL

## 22.5 Environment Variables

### Backend (Render) — 15 variables:

```
DATABASE_URL=postgresql://...(pooler, port 6543)...?pgbouncer=true
DIRECT_URL=postgresql://...(pooler, port 5432)...
JWT_ACCESS_SECRET=[random 64-char hex]
JWT_REFRESH_SECRET=[random 64-char hex]
JWT_ACCESS_EXPIRY=15m
JWT_REFRESH_EXPIRY=7d
NODE_ENV=production
PORT=5000
CLIENT_URL=https://virtual-time-capsule-ten.vercel.app
SMTP_HOST=smtp.gmail.com
```

```
SMTP_PORT=587
SMTP_USER=[app email]
SMTP_PASS=[app password]
FROM_EMAIL=[app email]
FROM_NAME=Virtual Time Capsule
```

### Frontend (Vercel) — 1 variable:

```
VITE_API_URL=https://virtual-time-capsule-iif2.onrender.com/api
```

### Known Production Limitations:

**Cold start:** Render free tier spins down after 15 minutes of inactivity. First request takes ~30 seconds. Auth context includes retry logic.

**SMTP blocked:** Render blocks port 587 outbound. Gmail SMTP emails timeout. Share links are the primary notification mechanism. Brevo HTTP API planned as a fix.

# 23. Local Development Setup

Prerequisites: Node.js (v18+), PostgreSQL (v14+), Git, VS Code

## Step-by-Step:

```
# 1. Clone the repository
git clone https://github.com/KAIRUPPALA-HEMACHANDRA/virtual-time-capsule.git
cd virtual-time-capsule

# 2. Set up the backend
cd server
npm install
npm install @prisma/client@6 --save-exact
npm install -D prisma@6 --save-exact

# 3. Create local database
psql -U postgres -c "CREATE DATABASE virtual_time_capsule;"

# 4. Configure environment
# Create server/.env with DATABASE_URL, DIRECT_URL, JWT secrets, etc.

# 5. Run migrations
npx prisma@6 migrate dev
npx prisma@6 generate

# 6. Start backend
npm run dev

# 7. Set up frontend (new terminal)
cd ../client
npm install
npm run dev

# 8. Open http://localhost:5173
```

## Switching Between Laptops:

```
cd virtual-time-capsule
git pull
cd server && npm install && npx prisma@6 migrate dev && npx prisma@6 generate
cd ../client && npm install
```

## Prisma Cache Fix (if "Unknown argument" errors):

```
cd server
rmdir /s /q node_modules\.prisma
rmdir /s /q node_modules\@prisma\client
npm install @prisma/client@6 --save-exact
npx prisma@6 generate
```

# 24. Challenges & Solutions

### BullMQ/Redis → pg-boss

**Problem:** Redis has no official Windows installer. BullMQ requires Redis.

**Solution:** Switched to pg-boss, which uses PostgreSQL as its job store. Same reliability, zero additional infrastructure.

### Prisma Version Compatibility

**Problem:** Prisma v7/v8 broke existing queries and migration commands.

**Solution:** Pinned to v6 using --save-exact. All commands use npx prisma@6.

### Prisma Client Cache Staleness

**Problem:** After schema changes, Prisma used old generated code causing "Unknown argument" errors.

**Solution:** Force-regenerate: delete node_modules/.prisma, run npx prisma@6 generate.

### pg-boss Job ID Format

**Problem:** pg-boss v12 requires UUID-format job IDs. Prefixed IDs ("unlock-" + id) caused validation errors.

**Solution:** Changed to use capsule UUID directly as job ID.

### JWT Token Collisions

**Problem:** Rapid sequential token generation (during tests) produced identical tokens with same payload and timestamp.

**Solution:** Added random JTI field: jti: Math.random().toString(36).substring(2).

### SMTP Timeout on Render

**Problem:** Gmail SMTP connections consistently timed out on Render free tier (port 587 blocked/rate-limited).

**Solution:** Implemented 3-retry with 30s timeout. Planned Brevo HTTP API migration. Share links serve as primary notification.

### Self-Destruct Owner Bug

**Problem:** Owner viewing a self-destruct capsule triggered destruction before recipient could see it.

**Solution:** Added !capsule.selfDestructAfterRead condition to owner view logic.

### SPA Routing on Vercel

**Problem:** Refreshing any non-root URL returned 404 — Vercel looked for actual files.

**Solution:** Added vercel.json: {"rewrites":[{"source":"/(.*)", "destination":"/index.html"}]}

### Trust Proxy Warning

**Problem:** express-rate-limit threw errors about X-Forwarded-For behind Render's proxy.

**Solution:** Added app.set("trust proxy", 1).

### Dashboard Infinite Loop

**Problem:** Socket notification count change triggered re-fetch which triggered re-render repeatedly.

**Solution:** Used useRef to track previous count, only fetch when count increases.

### Two-Laptop Development

**Problem:** Switching between personal and office laptops required consistent environments.

**Solution:** Git-based sync with documented setup commands. DIRECT_URL added to both .env files.

# 25. User Guide

## 25.1 Creating Your First Capsule

1. Register an account or log in at the landing page
2. Click "+ Create" in the navbar
3. Enter a title and write your message
4. Optionally attach files, record a voice memo, or record a video
5. Set the unlock date and time (when the capsule should open)
6. Choose any special options: geo-lock, chain, encrypt, legacy, anonymous, self-destruct
7. Add recipient emails if you want to send it to someone
8. Click "Seal Capsule" — your message is now locked until the unlock date

## 25.2 Sharing a Secret Message

1. Create a capsule with your secret message
2. Check "Send anonymously" to hide your identity
3. Optionally check "Self-destruct after reading" for one-time viewing
4. Set the unlock time (e.g., midnight tonight)
5. Seal the capsule and wait for it to unlock
6. Once unlocked, click "Copy Share Link"
7. Send the link via WhatsApp with a mysterious message: "Open this at midnight. Trust me."
8. The recipient opens the link, reads your anonymous message, and taps an emoji reaction
9. You receive a notification with their reaction — they never know who sent it

## 25.3 Creating a Treasure Hunt

1. Create Capsule 1: geo-lock it to Location A, write a clue leading to Location B
2. Create Capsule 2: geo-lock it to Location B, chain it to Capsule 1, write a clue for Location C
3. Create Capsule 3: geo-lock it to Location C, chain it to Capsule 2, write the final reveal
4. Share the link to Capsule 1 with the treasure hunter
5. They must go to Location A, open Capsule 1 (geo-check), read the clue
6. Then go to Location B, open Capsule 2 (geo-check + prerequisite check), and so on
7. Each capsule only opens at the right location AND after the previous one is opened

## 25.4 Digital Legacy Setup

1. Create a capsule with an important message for a loved one
2. Add their email as a recipient
3. Enable "Digital Legacy Mode"
4. Choose the inactivity period (30 days, 3 months, 6 months, or 1 year)
5. Seal the capsule — it will only be delivered if you stop logging in
6. Simply logging in resets the timer — the capsule stays sealed as long as you're active
7. If something happens and you're inactive beyond the threshold, the capsule is delivered automatically

# 26. Future Enhancements

| Enhancement | Description | Complexity |
|---|---|---|
| Docker Containerization | Dockerfile for backend/frontend + docker-compose.yml for one-command deployment | Medium |
| Brevo Email API | Replace SMTP with HTTP-based email (300 free/day) to solve Render timeout | Low |
| Calendar Heatmap | GitHub-style contribution graph showing capsule creation activity by day | Medium |
| Geo-Location Map | World map showing pins for geo-locked capsules using react-simple-maps | Medium |
| Web Push Notifications | Service worker-based push for instant alerts even with browser tab inactive | High |
| Custom Themes | Background colors, fonts, and moods for capsules — more personal and visual | Medium |
| Countdown Share Pages | Public countdown page for social media stories while waiting for unlock | Medium |
| Mobile App (PWA/React Native) | Native mobile experience with push notifications and camera | High |
| Content Moderation | Automated filtering for public capsule wall to prevent abuse | Medium |
| Cloud File Storage | Move from local Multer storage to S3/Cloudinary for persistent file hosting | Medium |
| Multi-Language Support | i18n for Hindi, Telugu, and other languages | Medium |

*End of Documentation*

Virtual Time Capsule v1.0 — September 2026

*Built with passion by Kairuppala Hemachandra*